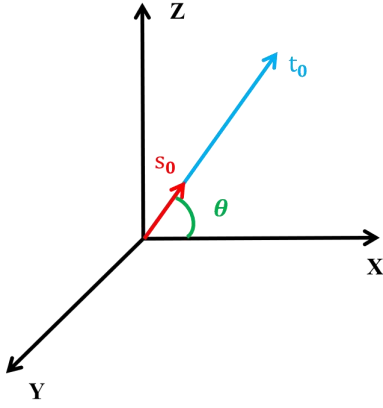
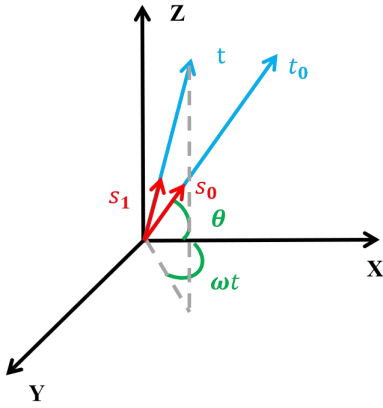


波束合成相位补偿



t_0 时刻，基线指向射电源的方向矢量为 s_0 ，且 s_0 在 XOZ 平面，夹角为 θ （天线俯仰角），此时方向矢量表示为：

$$\hat{\mathbf{s}}(t_0) = (\cos \theta, 0, \sin \theta)$$



在 t 时刻，射电源以角速度 ω_i 在 xoy 面转过角度 $\delta\varphi = \omega_i(t - t_0)$
此时方向矢量表示为：

$$\hat{\mathbf{s}}(t) = (\cos \theta \cos \delta\varphi, \cos \theta \sin \delta\varphi, \sin \theta)$$

t 时刻相比 t_0 时刻的方向变化量为：

$$\Delta\hat{\mathbf{s}} = \hat{\mathbf{s}}(t) - \hat{\mathbf{s}}(t_0)$$

代入上式

$$\Delta\hat{\mathbf{s}} = (\cos \theta \cos \delta\varphi - \cos \theta, \cos \theta \sin \delta\varphi - 0, \sin \theta - \sin \theta)$$

整理得

$$\Delta\hat{\mathbf{s}} = (\cos \theta (\cos \delta\varphi - 1), \cos \theta \sin \delta\varphi, 0)$$

几何延迟变化

设阵列位置基线向量为：

$$\mathbf{B} = (B_x, B_y, B_z)$$

则几何延迟为：

$$\tau(t) = \frac{\mathbf{B} \cdot \hat{\mathbf{s}}(t)}{c}$$

则 t 时刻相对于 t₀ 时刻的时延变化量为：

$$\Delta\tau(t) = \tau(t) - \tau(t_0) = \frac{\mathbf{B} \cdot (\hat{\mathbf{s}}(t) - \hat{\mathbf{s}}(t_0))}{c} = \frac{\mathbf{B} \cdot \Delta\hat{\mathbf{s}}}{c}$$

代入 $\Delta\hat{\mathbf{s}}$ ：

$$\Delta\tau(t) = \frac{l}{c} [B_x \cos \theta (\cos \delta \varphi - 1) + B_y \cos \theta \sin \delta \varphi]$$

因为第三分量为 0，所以 B_z 不起作用

相位变化量等于 频率 * 时延 * 2pi

$$\Phi_3(t) = 2\pi\nu \Delta\tau(t)$$

代入上式：

$$\Phi_3(t) = \frac{2\pi\nu}{c} [B_x \cos \theta (\cos \delta \varphi - 1) + B_y \cos \theta \sin \delta \varphi]$$

展开：

$$\Phi_3(t) = \frac{2\pi\nu}{c} B_x \cos \theta \cos \delta \varphi + \frac{2\pi\nu}{c} B_y \cos \theta \sin \delta \varphi - \frac{2\pi\nu}{c} B_x \cos \theta$$

提：

$$\Phi_3(t) = \nu \left[\frac{2\pi}{c} B_x \cos \theta \cos \delta \varphi + \frac{2\pi}{c} B_y \cos \theta \sin \delta \varphi - \frac{2\pi}{c} B_x \cos \theta \right]$$

令

$$A_i = \frac{2\pi}{c} B_x \cos \theta$$

$$B_i = \frac{2\pi}{c} B_y \cos \theta$$

可写为：

$$\Phi_3(t) = \nu [A_i \cos \delta \varphi + B_i \sin \delta \varphi - A_i]$$

其中 $\delta\varphi = \omega_i(t - t_0)$

参数更新协议:

(1) 以数据文件形式更新

有三类参数要更新, 对应着三个文件:

- ① time_phase_coeff.dat 存 时延补偿参数 τ_c 和相位修正因子 Φ_1 ;
- ② beam_coeff.txt 存 波束因子 Φ_2
- ③ trace_mode_phase_coeff.txt 存 跟踪模式相位因子 Φ_3

time_phase_coeff.dat 记录 时延补偿参数 τ_c 和相位修正因子 Φ_1 ;

记录:

每根天线的延时补偿参数 τ_c , 32bits 有符号定点数, 小数位 0bit, 4 的整数倍, 单位 ns;

每根天线、每个频点的相位修正因子 Φ_1 , 复数, 实部虚部各 16bits 有符号定点数, 小数位均 14bits;

两分钟更新一次。

数据文件格式:

小端存储, 先存 τ_c 再存 Φ_1

参数写入先记录每根天线的延时补偿参数:

$$\tau_{c_A1}, \tau_{c_A2}, \dots, \tau_{c_A4}$$

再记录 复数相位因子:

天线 1 的: $\Phi_{1_A1_F1}, \Phi_{1_A1_F2}, \dots, \Phi_{1_A1_F1024}$

天线 2 的: $\Phi_{1_A2_F1}, \Phi_{1_A2_F2}, \dots, \Phi_{1_A2_F1024}$

.....

天线 4 的: $\Phi_{1_A4_F1}, \Phi_{1_A4_F2}, \dots, \Phi_{1_A4_F1024}$

问题:

1、相位因子是通过实验补偿参数计算得到的吗? $2\pi\nu\tau_c$

小端存储: 一个多字节数字在内存或文件里保存时, 低位字节放前面, 高位字节放后面。

例: 32bit 整数, 0x12345678, 共四字节, 从高到低 12、34、56、78

小端存储, 低位在前 78 56 34 12

大端存储, 高位在前 12 34 56 78

beam_coeff.txt 记录 波束因子 Φ_2 ;

Φ_2 为三维矩阵 $N[i, j, \nu]$

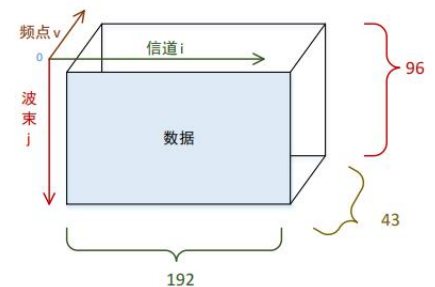
i : 天线信号通道标号, 数量为 4

j : 波束标号, 数量为 32

ν : 频率标号, 数量为 43, 与 GPU 数量有关 $43 * 24 = 1032$ 。

——>对每个通道 i , 波束 j , 频点 ν 都有一个对应的波束系数 $\Phi_2[i, j, \nu]$

——>每一个 GPU 的数据大小是 $4 \times 32 \times 43$, 总共 24 份。



问题: 24 个 GPU 的文件写入一个 beam_coeff.txt (记录波束因子 Φ_2), GPU 数量由什么决

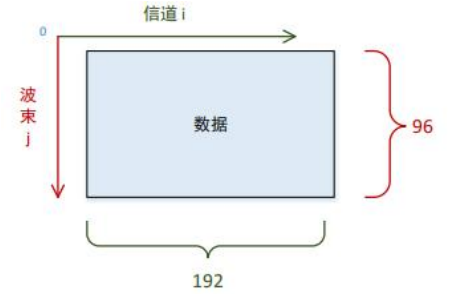
定，CARRY 使用多少个 GPU?

beam_coeff_antenna.txt 记录 波束开关 0 或 1;

- 0: 该天线的某个波束强制为 0
- 1: 按照 beam_coeff.txt 的设置进行使用

参数为二维矩阵 $M[i, j]$

- i : 天线信号通道标号, 数量为 4;
- j : 波束标号, 数量为 32。

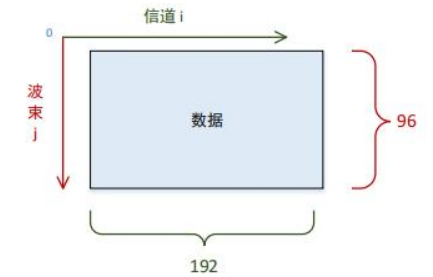


trace_mode_phase_coeff.txt 记录 跟踪模式相位因子 Φ_3 ;

按照 A_i, B_i, ω_i 三组参数 顺序存储。每组参数都是二维矩阵 :

$$M[i, j]$$

每组参数大小 $4 * 32$ 。



$$\Phi_3(t) = v [A_i \cos \delta \varphi + B_i \sin \delta \varphi - A_i]$$

$$A_i = \frac{2\pi}{c} B_x \cos \theta$$

$$B_i = \frac{2\pi}{c} B_y \cos \theta$$

$$\delta \varphi = \omega_i(t - t_0)$$

由于 $\Phi_3(t)$ 会随着时间变化, 所以工程上只要把 A_i, B_i, ω_i , 发给设备, 设备可以根据观测的起始时刻和结束时刻, 计算 $\Phi_3(t)$ 。

宋亚芳老师提供两个文件作为参考：

 time_phase_coeff(1).md5	2026/3/17 10:12	MD5 文件	1 KB
 time_phase_coeff(1).npz	2026/3/17 10:12	NPZ 文件	3,075 KB

npz 是 NumPy 的压缩数据文件，通常装着一个或多个数组。

md5 是对文件内容算出的哈希值，用于检验文件。

```
D:\总\博\imaging\uvcovplot-master\beamforming_delay_work>E:/Anaconda/python.exe d:/总/博/imaging/uvcovplot-master/beamforming_delay_work/read_npz_file.py
里面的键: ['coeff', 'time_delay', 'trace']

键名: coeff
shape: (192, 1024)
dtype: complex128
前几个值: [0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]

键名: time_delay
shape: (192,)
dtype: int64
前几个值: [ 0  4  8 12 16 20 24 28  0  4]

键名: trace
shape: ()
dtype: int64
前几个值: [0]

D:\总\博\imaging\uvcovplot-master\beamforming_delay_work>E:/Anaconda/python.exe d:/总/博/imaging/uvcovplot-master/beamforming_delay_work/read_npz_file.py
里面的键: ['coeff', 'time_delay', 'trace']

键名: coeff
shape: (192, 1024)
dtype: complex128
前几个值: [0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]

键名: time_delay
shape: (192,)
dtype: int64
前几个值: [ 0  4  8 12 16 20 24 28  0  4]

键名: trace
shape: ()
dtype: int64
前几个值: [0]
```

代码检查：compute_beamforming_reference_delay.py

run_forever()

- ① 读天线列表，天线序号 纬度 经度 海拔 天线直径；
ants, ant_names = load_antennas(ANTS_TXT)
- ② 设置目标源，输入赤经 赤纬；
target = katpoint.Target("FRB0525B, radec, {}, {}".format(TARGET_RA, TARGET_DEC))
[<name list>.] radec [<user tags>], <ra>, <dec> [, <flux model>]
print("target: {}".format(target))
target: FRB0525B, tags=radec, 19:35:00.00 21:54:00.0, no flux info
目标名：Target
类型：radec
赤经：ra
赤纬：dec
- ③ 生成频率轴；
frequencies_hz = build_frequency_axis_hz()

- ④ 确定程序第一次计算时延参数的 UTC 时间;
`solution_time_utc = resolve_first_solution_time()`
程序有两种启动方式,
A: 立刻开始, 不手动指定开始时间, 程序用当前 UTC 时间作为第一个解算时刻;
B: 指定时间, 将输入的时间字符串当做第一个解算时刻。

- ⑤ 循环计算, 是否开始计算时延参数;
`solution_time_utc = maybe_wait_for_start_window(target, ref_ant, solution_time_utc)`
`WAIT_FOR_START_WINDOW = False`
——>用当前 UTC 进入等待循环, `True` 用设定时间算
`REQUIRE_TARGET_VISIBLE = True` 要看着这个源才算
——>1)如果是全天在天上, 直接算; 2) 如果第一次 UTC 在窗口内, 直接算; 3)
还看不到, 等待循环

- ⑥ 算参考天线的俯仰角、方位角

```
def get_visibility_status(target, ref_ant, when_utc):  
    observer = make_observer_for_time(ref_ant, when_utc)  
    target.body.compute(observer)  
    alt_deg = float(target.body.alt) * 180.0 / np.pi  
    az_deg = float(target.body.az) * 180.0 / np.pi  
    is_visible = alt_deg >= VISIBILITY_HORIZON_DEG  
    return is_visible, alt_deg, az_deg
```

Target 里有目标源信息, 赤经赤纬

Observer 里有观测开始时刻、经纬度、海拔、地平线高度 (角度)

核心: `target.body.compute(observer)`, 将目标源的赤道坐标, 结合观测者时间和地址, 换算成观测者的地平坐标。

设定观测时间 `observer.date = ephemeris.Date(when_utc)` 第一轮计算的 UTC

读取地平线高度 `observer.horizon = str(VISIBILITY_HORIZON_DEG)`

调用函数: `target.body.compute(observer)`

取出结果并单位转换: `alt_deg = float(target.body.alt) * 180.0 / np.pi`

`az_deg = float(target.body.az) * 180.0 / np.pi`

计算过程:

从 UTC 和经度, 得到本地恒星时 LST;

由赤经算时角, $HA = LST - RA$, 知道当前本地子午线相对于目标赤经偏了多少;

用时角 HA、经纬度 Dec lat 算俯仰角 alt, 标准球面天文公式:

$$\sin(alt) = \sin(dec) \sin(lat) + \cos(dec) \cos(lat) \cos(HA)$$

再算方位角 az, 正北 0 度, 90 度东 180 度南 270 度西

再计算俯仰角 alt, 地平线是 0°, 头顶是 90°

不太懂地球自转 方向角随着时间咋变化, 靠北天极的 靠赤道的

```

Cycle 0000 | solution time 2026-03-19 20:07:49 UTC | target not visible at reference ant17 | alt 31.804 deg | az 81.261 deg | horizon 32.00 deg
next rise 2026-03-19 20:08:43 UTC | next set 2026-03-20 04:44:47 UTC | next transit 2026-03-20 00:26:45 UTC

observer.date: 2026/3/19 20:09:49
target.body: 32:14:04.0 81:28:10.7
target.body: 0.5625972747802734 1.4219132661819458
Cycle 0001 | solution time 2026-03-19 20:09:49 UTC | reference ant24
target visible at reference ant17 | alt 32.234 deg | az 81.470 deg | horizon 32.00 deg
Target pointing at 2026-03-19 20:09:49 UTC:
sky coordinates (fixed source): RA 19:35:00.00 | Dec 21:54:00.00
horizontal coordinates per antenna: alt/az in degrees
ant17 alt 32.234 deg | az 81.470 deg
ant20 alt 32.234 deg | az 81.469 deg
ant21 alt 32.234 deg | az 81.469 deg
ant24 alt 32.233 deg | az 81.469 deg
ant17 total 294.933 ns | time_delay 302 step (294.921875 ns) | fine 0.011028 ns
ant20 total 109.669 ns | time_delay 112 step (109.375000 ns) | fine 0.293607 ns
ant21 total 68.918 ns | time_delay 70 step ( 68.359375 ns) | fine 0.558609 ns
ant24 total 0.000 ns | time_delay 0 step ( 0.000000 ns) | fine 0.000000 ns
wrote 8 active signal inputs and 12 zero-padded signal inputs

```

若设置的地平线角度 大于 望远镜的俯仰角，则看不到源，随着源出来，望远镜俯仰角>地平线角度，继续输出参数文件。

⑦ 观测窗口

try:

```

if is_visible:
    rise = observer.previous_rising(target.body,
start=observer.date)
    set_ = observer.next_setting(target.body, start=observer.date)
    summary["window_contains_when_utc"] = True
else:
    rise = observer.next_rising(target.body, start=observer.date)
    set_ = observer.next_setting(target.body, start=rise)
    summary["rise_utc"] = rise.datetime()
    summary["set_utc"] = set_.datetime()

```

if is_visible:

计算这段可见窗口啥时候开始的

计算这段可见窗口啥时候结束

Else

下一次啥时候升到可见阈值上

下一次啥时候落

Transit 计算何时过中天（本地子午线），对应着天线方位角 正北或者正南

⑧ 计算几何延迟

几何延迟 = 路径差/光速

首先将第一面天线当做基准，然后对每一个其他天线，调用

`_, _, ww = target.uvw(internal_anchor, t_sec, ant)`

`delays_s[ant_names[i]] = ww / LIGHT_SPEED_M_PER_S`

其他天线分别与第一面天线组成基线，返回 (u,v,w) uv 用于 UV 平面的采样点，W 沿目标视线方向的几何分量，除以光速，转换为秒。

选取时间最长的那个值，第一面天线时间为 0 其余有正有负。

计算其余天线与参考天线的相对延迟。

以两个采样点的间隔时间作为单位，计算粗补偿数值。

相对延迟 - 粗补偿 = 细补偿。

$2\pi f * \text{细补偿}$ 得到每个频点的补偿数值。

beam_coeff.txt 中 记录每个频点每个天线的每个波束的因子。

天线波束数量由天线的硬件决定，指向是可以设置的。

波束形成 = 对阵列中各天线收到的信号，施加合适的延时或相位/权重，再把它们相加，使某个方向的信号同相叠加、增强，而其他方向的信号不能很好叠加、被抑制。

每面天线得到两路原始电压信号。

每路信号在数字后端复制成 32 路分支。

32 路分支在单天线层面 不是 32 个不同方向的波束。

阵列层面上：所有天线的第 1 路分支按权重相加，得到第 1 个波束；

所有天线的第 1 路分支按另一套权重相加，得到第 2 个波束；

...

最终得到 32 个阵列波束。

复制一路天线 32 份不会让单天线变成 32 个指向，把所有天线信号都复制 32 份，按 32 套权重跨天线求和，整个阵列可以形成 32 个波束。

假设阵列有 N 面天线，每面天线一路信号：

$$x_1(t), x_2(t), \dots, x_N(t)$$

对于某个天空方向 θ ，到第 n 面天线会有几何延时 $\tau_n(\theta)$ 。

如果一个源在方向 θ ，那么它在各天线上的信号大致是：

$$x_n(t) \propto s(t - \tau_n(\theta))$$

窄带近似下可写为：

$$x_n(t) \propto s(t) e^{-j2\pi v \tau_n(\theta)}$$

同一个源，到不同天线，相位不一样。

如果想让阵列“指向”某个方向 θ_k ，需要给第 n 面天线乘上一个补偿相位：

$$w_{k,n} \approx e^{+j2\pi v \tau_n(\theta_k)}$$

求和：

$$y_k(t) = \sum_{n=1}^N w_{k,n} x_n(t)$$

如果真实信号刚好来自 θ_k 方向，则相位补偿后会同相相加，信号变强。

如果来自别的方向，就不能完全对齐，相加后不强。

形成第 k 个阵列波束。

1 路信号复制 32 份的意义是，给 32 个不同的阵列波束各准备一套独立的加权支路。

32 个波束的方向，本质是 32 个波束中心在天空上的位置。

- 1) 单天线主波束有多大？决定 32 个波束最多能铺多大一块天区。
- 2) 阵列合成后的单个窄波束有多宽？决定相邻两个波束中心该隔多远。

单天线波束宽度

$$\theta_{pb} \approx 1.22 \times \frac{\lambda}{D} \approx 2.24^\circ$$

中心频率 1.25GHz，天线直径 7.5m，32 个阵列波束可以设置的范围。

阵列合成波束宽度

阵列波束合成后输出的窄波束

$$\theta_{syn} \approx 1.22 \times \frac{\lambda}{B} \approx 0.15^\circ$$

决定 32 个波束中心之间该如何排，间隔多大。

方向如何设置？

——>在主波束允许的视场里，给 32 个阵列波束选取 32 个中心位置。

波束方向设置需要考虑半高全宽 FWHM。

——>半高全宽代表着波束的宽度。如果相邻波束中心间隔太大，会出现中间区域灵敏度掉的多，重则漏源。如果间隔太小，会出现重叠太多，32 个波束浪费一些，相邻波束的相关性增强。

波束中心间隔，本质上在平衡覆盖连续性和覆盖面积。

一面碟形天线，有 3 个不同的角度量：

$$\frac{\lambda}{D}, 1.02 \frac{\lambda}{D}, 1.22 \frac{\lambda}{D} \text{ 真的吗?}$$

都和波束有关，但是含义不同。

$$\lambda/D \approx 1.83^\circ$$

$$1.02\lambda/D \approx 1.87^\circ$$

$$1.22\lambda/D \approx 2.24^\circ$$

半功率全宽：从峰值下降到 50% 的两个点之间有多宽。

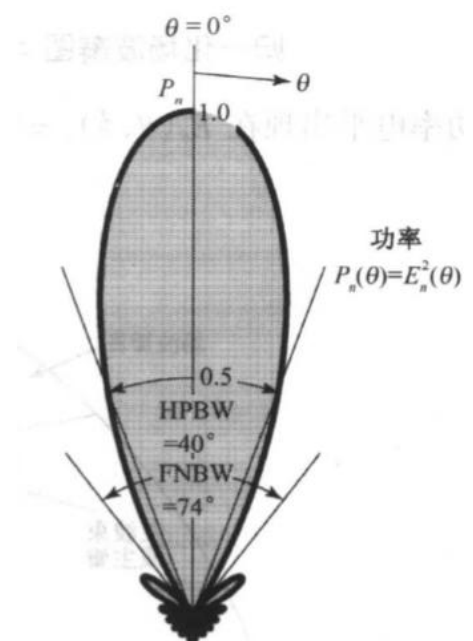
第一零点，从中心第一次讲到 0 的位置有多远。

中间最高点 = 波束中心

两个“0.5 的点”之间的宽度 = FWHM

从中心到第一次掉到 0 的位置 = 第一零点角

左右两个零点之间的总宽度 = 零点到零点宽度



初步方案

1、基本尺度

观测中心频率 1.25GHz，波长

$$\lambda = \frac{c}{f} \approx 0.2398 \text{ m.}$$

单天线口径 $D = 7.5 \text{ m}$ 。单天线主波束的半波全宽近似取：

$$\theta_{PB,FWHM} \approx 1.02 \frac{\lambda}{D}$$

得到大约 1.87° 。

第一零点角度则约 2.24° （中心到第一零点的角度），主瓣宽度为 $2 \times 2.24^\circ = 4.48^\circ$ 。

因此单天线主波束的半功率半径约 $56.1'$ （半功率全宽除 2）。

阵列波束合成

$$\theta_{syn} \approx \frac{\lambda}{B_{max}}$$

估算： $\theta_{syn} \approx 0.1195^\circ \approx 7.17'$ ，波束合成后，32 个阵列波束的单波束宽度量级。

指定波束的中心间距为 0.5HPBW ，代入

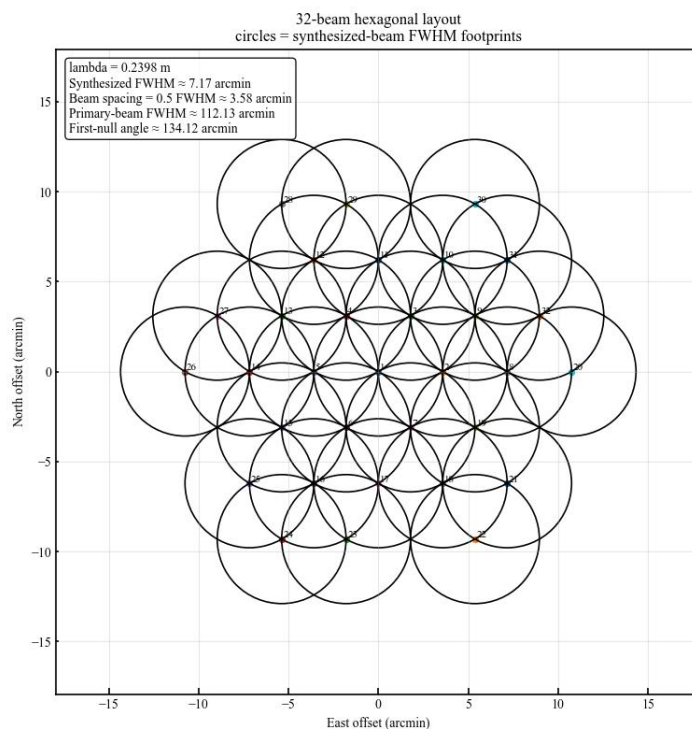
$$s = 0.5 \theta_{syn} = 0.5 \times 7.17' = 3.585'.$$

若选择六角排布，则六角排布行间距为：

$$\Delta y = \frac{\sqrt{3}}{2} s \approx 3.104'.$$

六角晶格来排布，最近邻中心距离为 $3.585'$ ，相邻两行的垂直距离为 $3.104'$ 。

当前 32 个波束组成的



先定摆放，再计算观测天区大小。

FRB 要求大视场 更大的天空面积，但是如果波束拉的太开，会有束间空洞，灵敏度低谷，边缘增益太差，

在视场和灵敏度之间找折中。

波束跟踪模式下的相位因子

$$\Phi_3(t) = v [A_i \cos \delta \varphi + B_i \sin \delta \varphi - A_i]$$

其中 $A_i = \frac{2\pi}{c} B_x \cos \theta$ 、 $B_i = \frac{2\pi}{c} B_y \cos \theta$ 、 $\delta \varphi = \omega_i(t - t_0)$

——>相位因子与源的角速度 ω_i 及 射电望远镜的俯仰角有关。

地球自转，看起来源在动，时角随着时间变化：

$$H(t) = \text{LST}(t) - \alpha$$

LST 本地恒星时， α 目标源的赤经。

固定在天球上的天体，地球自转带来的基本角速度是一样的，所有固定天体的时角变化率都一样。

时角反映天体自经过当地子午线以来的角距离。

求角速度

$$\frac{dH}{dt} = \omega_{sid} = \frac{2\pi}{86164 \text{ s}}$$

10s 内的变化大约是：

$$\Delta H \approx \omega_{sid} \cdot 10 \approx 0.04178^\circ$$

将时角变化转化为“天空里转了多少度”——与赤纬有关

源在“固定赤纬的小圆”上运动，因此 10s 内在天球上扫过的真实角距离

Estimating ASKAP beam to beam correlation

D McConnell
CSIRO Astronomy and Space Science
ACES memorandum 014

8th March 2017

真正做设计时，你要按这个顺序想

第一步：先算单天线主波束

这决定 32 个波束最多能铺多大。
你的这个量级大约是 2° 左右。

第二步：算阵列合成波束宽度

这才是决定相邻束间隔的关键量。
如果不知道阵列最长基线 B ，这一步还不能定死。

第三步：决定你的目标

你是要：

- 尽量不漏源
- 尽量覆盖更大面积
- 还是中心区灵敏度尽量均匀

不同目标，对间隔要求不同。

第四步：选排布

通常优先选 六角排布。

第五步：定中心距

一般先拿

$$s \approx 0.5 \sim 0.7 \theta_{\text{syn}}$$

做第一版设计。

跟踪模式下，一次观测周期为 20 分钟，每 10 秒钟更新一次参数文件，参数为相位补偿因子。

计算相位补偿因子前，需选择参考天线（通常选取平面波最晚到达的天线），其余天线计算相对于参考天线的延迟。

问题：如果在一次观测周期中，经过一段时间后，改变了参考天线，对系统输出有什么影响？

设目标源是平面波，第*i*个天线接收到的信号写为：

$$x_i(t) = s(t - \tau_i)$$

$s(t)$ 是源信号， τ_i 是几何到达延迟。

首先选取天线*r*为参考天线（平面波最晚到达， τ_r 最大），则对第*i*路施加补偿延迟为：

$$d_i^{(r)} = \tau_r - \tau_i$$

第*i*路补偿后，信号为： $x_i(t - d_i^{(r)})$ ，代入得：

$$x_i(t - d_i^{(r)}) = s(t - d_i^{(r)} - \tau_i)$$

再代入 $d_i^{(r)} = \tau_r - \tau_i$

$$x_i(t - d_i^{(r)}) = s(t - (\tau_r - \tau_i) - \tau_i) = s(t - \tau_r)$$

因此每路天线进行补偿后，理想情况下都会变成同一个信号：

$$s(t - \tau_r)$$

波束合成会对每路天线输出进行求和，写成：

$$y^{(r)}(t) = A s(t - \tau_r)$$

其中 $A = \sum_i w_i$ 是个常数。

当源动到某个位置的时候，天线*q*接收到的平面波最晚到达，在某个周期更新参数文件时，选取天线*q*作为参考天线。

对第*i*个天线新的补偿延迟为：

$$d_i^{(q)} = \tau_q - \tau_i$$

同理，补偿后的每一路信号都为：

$$s(t - \tau_q)$$

对每路天线信号求和，结果为：

$$y^{(q)}(t) = A s(t - \tau_q)$$

比较新旧参考天线下的输出：

旧输出： $y^{(r)}(t) = A s(t - \tau_r)$

新输出： $y^{(q)}(t) = A s(t - \tau_q)$

联系：

$$y^{(q)}(t) = A s(t - \tau_q) = A s((t - (\tau_q - \tau_r)) - \tau_r)$$

$$y^{(q)}(t) = y^{(r)}(t - (\tau_q - \tau_r))$$

定义： $\Delta\tau = \tau_q - \tau_r$

则：

$$y^{(q)}(t) = y^{(r)}(t - \Delta\tau)$$

输出可以写为分段形式：

$$y(t) = \begin{cases} y^{(r)}(t), & t < t_s \\ y^{(r)}(t - \Delta\tau), & t \geq t_s \end{cases}$$

系统在 $t < t_s$ 采用旧参考，在 $t \geq t_s$ 时采用新参考。

从频域上看

$$y_{new}(t) = y_{old}(t - \Delta\tau)$$

频域中：

$$Y_{new}(\nu) = Y_{old}(\nu)e^{-j2\pi\nu\Delta\tau}$$

对于单频窄带 f_0 ：

$$\Delta\phi = -2\pi f_0 \Delta\tau$$

对波束合成系统输出的功率谱有什么影响？

$$P(\nu) = |Y(\nu)|^2$$

更换参考天线后：

$$P_{new}(\nu) = |Y_{old}(\nu)e^{-j2\pi\nu\Delta\tau}|^2 = |Y_{old}(\nu)|^2$$

因此

$$P_{new}(\nu) = P_{old}(\nu)$$

理想情况下，切换参考天线对于波束合成系统输出的功率谱没影响。

系统通过功率谱识别到可能有 FRB 信号，触发基带数据保存功能，参考天线改变，对基带数据有影响吗？

假设采样间隔是 T_s ，则 ADC 输出的信号为：

$$y[n] = y(nT_s)$$

第 n 个采样点值，等于模拟信号中的第 n 个采样间隔的值。

参考天线变化，会导致切换点前后可能不是连续微调，不是连续读取采样点，相当于：

$$y_{new}[n] \approx y_{old}(nT_s - \Delta\tau)$$

如果 $\Delta\tau = mT_s$ ，恰好是整数倍采样点，则

$$y_{new}[n] = y_{old}[n - m]$$

这段数据整体平移 m 个采样点。

如果 $\Delta\tau$ 不是整数倍采样点， $\Delta\tau = (m + \alpha)T_s$ ， $0 < \alpha < 1$

等价于整体平移 m 个采样点，然后在频域里进行调整。

什么时候会对系统有影响？

切换参考天线的时候 $\tau_q(t_s) \neq \tau_r(t_s)$, $\Delta\tau = \tau_q(t_s) - \tau_r(t_s)$

当切换时满足： $\tau_q(t_s) = \tau_r(t_s)$ ， $\Delta\tau = 0$ ，则 $y^{(q)}(t) = y^{(r)}(t)$

先将这个 $\Delta\tau = \tau_q(t_s) - \tau_r(t_s)$ 称作 公共时间台阶，因为每路信号都要变一个时间延迟。

公共时间延迟的本质是：

切换参考天线时，整个阵列输出的时间参考零点从 τ_r 变成了 τ_q 。如果两者不同，输出就会整体平移：

$$\Delta\tau = \tau_q - \tau_r$$

相位追踪模式下 相位因子 计算流程:

ω 计算

先取前后两个时刻

`minus_dt = epoch_time_utc - timedelta(seconds=OMEGA_DELTA_SECONDS)`

`plus_dt = epoch_time_utc + timedelta(seconds=OMEGA_DELTA_SECONDS)`

默认 $\Delta t = 1$

分别计算三个时刻的 beam 状态

`state_minus`

`state_tn`

`state_plus`

这些状态里包含了每根天线、每个 beam 的方向向量。

建立局部坐标系

对当前时刻的 beam direction `beam_dir_tn`, 代码会构造:

`x_hat`

`y_hat`

`z_hat`

也就是一个“围绕当前 beam 指向”的局部坐标系。

将前后时刻的 beam 方向投影进去, 得到两个角

`phi_minus = compute_local_phase_angle_rad(...)`

`phi_plus = compute_local_phase_angle_rad(...)`

这两个角, 就是前后时刻在这个局部平面里的方位角。

做中心差分

$$\omega \approx \frac{\phi(t + \Delta t) - \phi(t - \Delta t)}{2\Delta t}$$

角速度计算考虑了目标源的赤经赤纬、观测时刻, 以及天线的空间位置;

先在 $t - \Delta t$ 、 t 、 $t + \Delta t$ 分别计算 beam 方向, 再把前后两个时刻的方向变化写成相位角变化, 最后用中心差分得到当前时刻的 ω 。

先算一个 $\omega_{ant, beam}$

4 根天线 x 32 个波束的二维数组, `omega_ant.shape = (4, 32)`

1. 对每根天线循环 `ant_index`

2. 对每根天线里面, 每个 beam 循环, `beam_index`

3. 对某根天线的某个 beam, 分别取 $t - \Delta t$ 、 t 、 $t + \Delta t$ 的方向

4. 算这一格 `omega_ant[ant_index, beam_index]`

总共得到 128 个 ω 。

将每根天线复制成 2 路有效信号, 前 8 行有值, 剩下补零, 也就是同一根天线的两路信号用一样的 ω 。

不同天线之间: ω 分别算, 不同波束之间: ω 分别算。

同一根天线复制出来的两路信号: ω 一样, 只是为了匹配输出通道格式。

波束布局

beam32_layout.py 里先定义了两组六边形坐标

CORE19: 中心 19 个

OUTER13: 外圈 13 个

两组坐标: CORE19 = [(0,0), (1,0), (0,1), (-1,1), ...] OUTER13 = [(3,0), (3,-2), (3,-3), ...]

所以每个波束先有一个整数坐标 (q, r)

(q, r) 可以理解成: 它不是“真实天空中的角度”, 它只是“第几个六边形格子”的编号

晶格间距

计算波长: $\lambda = \frac{c}{f_0}$

单天线的半高全宽: $\theta_{pb} \approx 1.02 \frac{\lambda}{D}$

阵列波束的半高全宽: $\theta_{syn} \approx \frac{\lambda}{B_{max}}$

B_{max} : 投影基线最大值

间隔: $spacing_rad = spacing_factor \times \theta_{syn}$

可以手动设置 DEFAULT_SPACING_FACTOR = 2, 调整间距

六边形坐标 (q, r) 怎么变成“偏离主波束多少角度”

先变东-北平面上两分量

build_beam_rows() 里, 每个 beam 的 (q, r) 会变成:

$$d_{East} = spacing_rad (q + 0.5r)$$

$$d_{North} = spacing_rad \left(\frac{\sqrt{3}}{2} r \right)$$

标准六边形轴坐标转二维平面坐标。

$$算总偏离角度: offset_rad = \sqrt{d_{East}^2 + d_{North}^2}$$

beam 相对 Beam 0 (主波束中心) 在切平面上的偏离角大小
角位置 PA

$$PA = \text{atan2}(d_{North}, d_{East})$$

转成 0-360 度, 该 beam 从主波束触发, 朝哪个方向偏过去, 东 0 北 90 西 180 南 270

举例:

Beam 0

$$(q, r) = (0, 0), d_E = 0, d_N = 0, offset = 0 \text{ 主波束本身}$$

Beam 1

$$(q, r) = (1, 0), d_E = spacing, d_N = 0, \text{ 它就在主波束正东边一个 spacing。}$$

spacing 就是: 相邻两个 beam 中心之间的角距离

Beam 2

$$(q, r) = (0, 1) d_E = 0.5 spacing d_N = \frac{\sqrt{3}}{2} spacing \sqrt{(0.5)^2 + \left(\frac{\sqrt{3}}{2}\right)^2} = 1$$

因此总偏离角度还是一个 spacing, 也就是因子乘半高全宽, 方向是 60 度

例 向东 3 米、向北 4 米 总距离是 5 米 总距离还是一个 spacing

偏离角度变成真实天空方向:

dEast、dNorth, 还只是“相对主波束的微小偏移”。

先求主波束的方向, compute_beam_vectors_for_antenna() 里, 先对某一根天线算主波束 Beam 0 的方向:

target.azel(...)

target.uvw_basis(...)

然后得到当前这个时刻、这根天线的局部基:

u_hat

v_hat

w_hat

其中你可以把:

w_hat 理解为主波束中心指向

u_hat、v_hat 理解为主波束附近天空切平面的两个正交方向

每个 beam 都有: dEast_rad dNorth_rad

构造 $tangent_offset = d_E u + d_N v$

生成新的 beam 方向

$$beam_hat = \cos(\rho) w_hat + \frac{\sin(\rho)}{\rho} tangent_offset$$

$$\rho = \sqrt{d_E^2 + d_N^2}$$

不是简单在方位角/高度角上硬加一个角度, 而是在主波束指向的局部切平面里, 按照东/北 两个小角偏移, 再映射回单位球面, 得到真实 beam 指向。

会判断 32 个波束有没有越出单天线主波束范围

$$\theta_{pb} \approx 1.02 \frac{\lambda}{D}$$

落在

$$\theta_{pb}/2$$

那你实际看代码时要注意什么？

1. 看 layout 文件时，重点看 `offset_deg`

因为它回答的是：

“这 32 个 beam 设计上是怎么摆的？”

也就是排布半径、第一圈第二圈、离中心多远这些问题，主要看 `beam32_layout.py` 里的结果。

`beam32_layout`

2. 看主脚本时，重点看 `separation_from_beam0_deg`

因为它回答的是：

“这一时刻、这一根天线上，真正算出来的 beam 和主波束差多少角？”

这更接近真实指向。 `trace_32beam_phase_coeff`

3. 不要把“东偏/北偏”误解成“直接加到 az/el 上”

`dEast_deg`、`dNorth_deg` 不是说：

- az 直接加一个数
- el 直接加一个数

代码不是这么干的。它是先在主波束局部坐标系里施加小偏移，再映射回单位球面，最后才得到真实方向。

这点很重要，因为很多人第一次看会误会成“就是把方位角高度角平移一下”。

你现在可以把它记成一句最简单的话

`beam32_layout.py` 里的 `offset_deg` 是“设计图纸上的偏移半径”；
主脚本里的 `separation_from_beam0_deg` 是“真正生成天空方向后量出来的实际夹角”。

参数输出

UPDATE_PERIOD_SECONDS = 10

PUBLISH_LEAD_SECONDS = 2.0

PRECOMPUTE_QUEUE_SLOTS = 4

参数时刻每 10s 一个，写文件提前 2s，提前算 4 个 slot 的参数。

slot 1 对应 12:00:20，大约 12:00:18 发布

slot 2 对应 12:00:30，大约 12:00:28 发布

程序启动后流程

- 1、读取天线文件 解析 4 面天线
- 2、生成 32 波束 offset 表，记录 `trace_32beam_beam_offsets.txt`
- 3、建立 `beam_coeff` 矩阵
- 4、启动预计算线程
- 5、启动发布线程
- 6、持续循环

预计算线程

`precompute_worker()` 会不停生成未来 slot 的 `SlotPackage`，塞进一个最多容纳 4 个 slot 的队列里。这个包里已经包含了：

slot 索引

该 slot 的 UTC 时刻

主波束 `az/el`、`LST`、`JD`

可见性状态

`beam_coeff_mat`

`a_mat`

`b_mat`

`w_mat`

每根天线的 32 beam 方向结果

写日志要用的信息

也就是说，发布线程不负责“重算”参数，它主要负责“按时拿出来写”。

发布线程：

`publisher_worker()` 会盯着当前应该发布哪个 slot。

对每个 slot，它先算：

`slot_epoch_utc`

`ready_time_utc = slot_epoch_utc - publish_lead_seconds`

没到 ready time 就等；到了 ready time 就从队列里取出该 slot 的包。然后：

先写 `trace_32beam_beam_directions.txt`

如果 `package.should_publish` 为真，就写两个主参数文件

追加一条 JSON 日志

在终端打印一条 slot 更新信息

这里有个很重要的点：

每个 slot 都会写方向报告和日志，但不一定每个 slot 都会更新两个主参数文件。

什么时候会“真正发布参数文件”

核心判断在 `build_slot_package()` 里：

`visible_now` = 当前源高度是否高于 `MIN_ELEVATION_DEG`

`forced_compute` = 不可见时是否强制计算

`should_publish` = `visible_now` or `forced_compute`

而当前默认配置是：

`MIN_ELEVATION_DEG = 32.0`

`FORCE_COMPUTE_WHEN_INVISIBLE = False`

所以默认逻辑其实是：

只有当源在参考天线看来可见，并且高度角 $\geq 32^\circ$ 时，才会刷新两个主参数文件。

如果源不可见：

`a_mat` / `b_mat` / `w_mat` 会保留初始化的零矩阵

但由于 `should_publish = False`，发布线程不会调用 `publish_slot_files()`

所以这三个零矩阵不会写进主参数文件
主参数文件会停留在上一次成功发布的内容
但方向报告和日志仍然会继续更新，并且日志动作会记成 `skipped_invisible`

主输出文件到底有哪些
程序里定义了 5 个主要输出路径：

`beam_coeff_antenna.txt`
`trace_mode_phase_coeff.txt`
`trace_32beam_beam_offsets.txt`
`trace_32beam_beam_directions.txt`
`trace_32beam_phase_coeff.log`

`beam_coeff_antenna.txt`
这个文件名字虽然叫 `.txt`，但实际上是二进制 `little-endian float32` 文件，因为写它用的是 `write_float32_le_temp_file()`。

它里面写的是 `package.beam_coeff_mat`。
这个矩阵是固定的 20×32 ，由 `build_beam_coeff_matrix()` 构造：

前 8 行全部是 1
后 12 行全部是 0

因为：

4 根天线
每根天线 2 路信号
所以前 8 路是 `active channels`

总输出通道预留到 20 行

所以这个文件本质上是一个固定的“使能/权重模板”，不是每个 `slot` 都重新算出新图案；它每次发布时写进去的内容其实相同

`trace_mode_phase_coeff.txt`
这个文件同样不是文本，也是 `binary little-endian float32`。
它的内容是把这三个 20×32 矩阵按顺序拼接后 `flatten`：

`a_mat`
`b_mat`
`w_mat`

代码就是：

先 `package.a_mat.reshape(-1)`
再 `package.b_mat.reshape(-1)`
再 `package.w_mat.reshape(-1)`
然后 `np.concatenate(...)` 写出

所以它的逻辑顺序是：

`[A 的  $20 \times 32$ ] + [B 的  $20 \times 32$ ] + [omega 的  $20 \times 32$ ]`

总长度是：

$3 \times 20 \times 32 = 1920$ 个 `float32`。

trace_32beam_beam_offsets.txt

这个文件是在程序启动阶段写一次的，不是每 10 秒都更新。

它记录的是：

当前 session 的 beam offset 来源

32-beam 固定偏移表

t_ref 时刻 4 根天线各 beam 的点位情况

也就是一个“本次运行的 beam 布局基准报告”

trace_32beam_beam_directions.txt

这个文件是每个 slot 都重写一次的。

发布线程在每个 slot ready 时，无论是否真正发布参数，都会先调用 write_beam_direction_report(package)。

里面是该 slot 的人类可读报告，包括：

当前 slot 索引和时间

visible_now / forced_compute / should_publish

每根天线的 beam0 az/el

u_hat / v_hat / w_hat

32 个 beam 的 q/r/dEast/dNorth/offset/sep/az/el/east/north/up

所以这个文件更像“运维/排查用的快照报告”。

依赖链

1. 在 `compute_trace_mode_phase_coeff.py` (line 959) 的 `resolve_beam_offset_table()` 里, 如果你没有提供 `BEAM_OFFSET_TABLE_FILE` 或 `BEAM_OFFSET_TABLE`, 就会走默认路径。
2. 默认路径会进入 `compute_trace_mode_phase_coeff.py` (line 838) 的 `build_default_beam_offset_table()`
3. 这里先算当前目标在 `t_ref` 时刻的投影最大基线 `projected_bmax`
4. 然后把
 - `f0_hz = CENTER_FREQ_HZ`
 - `dish_diameter_m = antennas[0].diameter_m`
 - `bmax_m = projected_bmax["projected_baseline_m"]`
 - `spacing_factor = beam32_layout.DEFAULT_SPACING_FACTOR`
传给 `beam32_layout.build_beam_rows(...)`
5. `beam32_layout.py` (line 57) 里定义了 32 个波束的六边形坐标 `HEX_COORDS`, 再在 `beam32_layout.py` (line 126) / `beam32_layout.py` (line 172) 里算出每个波束的:
 - `dEast_deg`
 - `dNorth_deg`
 - `offset_deg`
 - `position_angle_deg`
 - 以及 `spacing_deg`、`theta_syn_fwhm_deg`、`theta_pb_fwhm_deg` 等几何量
6. 这些 `rows` 就成了 `beam_offset_rows`, 下游在 `compute_trace_mode_phase_coeff.py` (line 997) 的 `compute_beam_model_state()` 和 `compute_trace_mode_phase_coeff.py` (line 1040) 的 `compute_abomega()` 里继续使用, 最终影响每个 beam 的方向、A/B/omega、以及输出文件内容。

一句话概括

`beam32_layout.py` 决定的是"32 个 beam 在主波束周围怎么排、每个 beam 相对 Beam 0 偏多少";

`compute_trace_mode_phase_coeff.py` 再拿这个几何排布去算每个时刻的 beam 指向和相位参数。

这会影响什么

如果你改 `beam32_layout.py` 里的这些内容, `compute_trace_mode_phase_coeff.py` 的结果就会变:

- `HEX_COORDS`: 32 波束编号和排布形状变
- `DEFAULT_SPACING_FACTOR`: 波束间隔变
- `compute_layout_metrics()`: spacing、合成波束宽度、主波束检查阈值变
- `build_beam_rows()`: 最终 `dEast/dNorth` 全部变

什么时候这个依赖不会生效

如果你在 `compute_trace_mode_phase_coeff.py` (line 959) 里提供了:

- `BEAM_OFFSET_TABLE_FILE`
- 或 `BEAM_OFFSET_TABLE`

那脚本会直接用你提供的 32 波束偏移表, 不再调用

`beam32_layout.build_beam_rows()`。



这时 `beam32_layout.py` 仍然被 import, 但默认几何生成这条依赖链就被绕开了。

beam_coeff 流程

- 1、输入天线 方位角 俯仰角
- 2、读取天线文件
- 3、根据当前时刻，计算 UTC 时刻，将方位俯仰 换算成 指向的赤经赤纬
- 4、获取 32 波束排布
- 5、根据投影基线长度计算 32 波束相对于主波束偏离角度
- 6、计算每个波束相对于参考天线对应波束的几何延迟
- 7、将几何延迟换算为相位参数
- 8、输出相位参数

trace_mode_phase_coeff 流程

- 1、给定要追踪的源的赤经赤纬
- 2、读取天线文件
- 3、设置间隙，判断源是否可见
- 4、根据天线的经纬度，源的位置，计算出天线方位角 俯仰角
- 5、根据方位俯仰角、源的位置、得到主波束指向
- 6、获取 32 波束排布文件，根据阵列最长投影基线长度计算 32 波束相对于主波束偏离角度
- 7、通过偏离角度计算每个波束的指向
- 8、计算每个天线 每个波束的 $A B \omega$ 参数
- 9、重复操作，系统每次计算出 x 次结果，间隔 x 秒输出下一次结果
- 10、输出参数

给定起始时间 计算参数。

给定起始时间 可见直接算，若不可见 给定的意义。